

OKU

Real-Time Social Media Platform

PROJECT REPORT

Built on the MERN stack with Socket.IO

August 2026

Table of Contents

- 1** Introduction
- 2** Objectives
- 3** Technology Stack
- 4** System Architecture
- 5** Database Schema
- 6** Core Features
- 7** API Reference
- 8** Real-Time Events (Socket.IO)
- 9** Security Measures
- 10** Challenges & Solutions
- 11** Future Enhancements
- 12** Conclusion

1. Introduction

Oku is a full-stack, real-time social media web application built on the MERN stack (MongoDB, Express, React, Node.js) augmented with Socket.IO for live, bidirectional communication. It provides the core mechanics of a modern social network: user accounts, a personalized feed, media-rich posts, likes and comments, follow relationships, direct messaging, and live notifications.

The application is organized as two independently deployable services — a React single-page frontend and an Express REST + WebSocket backend — sharing a MongoDB database and an AWS S3 bucket for media storage. This report documents the system’s architecture, data model, feature set, API surface, real-time event design, and the security measures applied throughout.

2. Objectives

The project set out to:

- Build a complete, production-shaped social platform end-to-end — frontend, backend, and database.
- Implement secure authentication and authorization using JSON Web Tokens (JWT).
- Support rich media posts and attachments (images and video) via cloud object storage.
- Provide real-time messaging, presence, and notifications using WebSockets.
- Apply industry-standard security practices: password hashing, rate limiting, input validation, and secure HTTP headers.
- Deliver a distinctive, custom-designed interface rather than a templated or generic theme.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, React Router 7, Vite	Single-page application, client-side routing, dev/build tooling
Frontend	Axios	REST API communication
Frontend	Socket.IO Client	Real-time messaging, notifications, presence
Frontend	Lucide React	Icon set
Backend	Node.js, Express 4	HTTP server and REST API routing
Backend	Mongoose 8	MongoDB object modeling (ODM)
Backend	Socket.IO	WebSocket server for real-time events
Backend	jsonwebtoken, bcryptjs	Authentication and password hashing

Layer	Technology	Purpose
Backend	Multer + AWS SDK v3 (S3)	Media upload handling and cloud storage
Database	MongoDB (Atlas)	Primary data store
Security	Helmet, CORS, express-rate-limit, express-validator	HTTP hardening, rate limiting, input validation
Tooling	Nodemon, oxlint, mongodb-memory-server	Dev auto-restart, linting, zero-setup local DB

4. System Architecture

Oku follows a three-tier architecture: a React client, an Express + Socket.IO application server, and a MongoDB data layer, with AWS S3 as a dedicated media store sitting alongside the database.

4.1 Request — Response Flow

- The React client communicates with the backend over two channels: REST (via Axios) for CRUD operations, and a persistent Socket.IO WebSocket connection for real-time events.
- Every authenticated REST request carries a JWT as a Bearer token; a protect middleware verifies the signature and attaches the resolved user to `req.user` before the route handler runs.
- Media uploads (post images/video, avatars, cover photos, message attachments) are received as multipart form data, buffered in memory by Multer, validated for type and size, then streamed directly to an AWS S3 bucket. Only the resulting S3 URL is persisted in MongoDB.
- On connection, the Socket.IO client emits a setup event with the user's ID; the server tracks a live `userId` → `socketId` map used to route messages, typing indicators, and notifications to the correct connected client, and to broadcast online-presence changes.

4.2 Component Responsibilities

Component	Responsibility
React SPA (frontend/)	Routing, UI rendering, client-side state (Auth, Socket, Toast contexts), optimistic UI updates
Express API (backend/)	Route handling, validation, business logic, JWT issuance/verification
Socket.IO server	Real-time message/notification delivery, typing indicators, online presence tracking
MongoDB (Atlas)	Persistent storage for users, posts, comments, messages, notifications
AWS S3	Durable object storage for all user-uploaded media

5. Database Schema

The database consists of five MongoDB collections, modeled with Mongoose schemas and linked via ObjectId references. All collections use Mongoose's automatic timestamps (`createdAt`, `updatedAt`).

5.1 User

Field	Type	Notes
username	String	Required, unique, lowercase, trimmed, min 3 characters
email	String	Required, unique, lowercase, trimmed, regex-validated
password	String	Required, min 6 characters, bcrypt-hashed; stripped from all JSON responses
profilePicture	String	S3 URL, defaults to empty string
coverPicture	String	S3 URL, defaults to empty string
bio	String	Max 160 characters
followers	[ObjectId] → User	Users following this user
following	[ObjectId] → User	Users this user follows
isAdmin	Boolean	Default false; reserved for future moderation tooling
lastSeen	Date	Updated on socket disconnect, used for presence display

5.2 Post

Field	Type	Notes
userId	ObjectId → User	Required, the post author
content	String	Max 2000 characters
media	String	S3 URL of attached image/video
mediaType	String enum	'image' 'video' 'none'
likes	[ObjectId] → User	Users who liked the post
commentsCount	Number	Denormalized counter, default 0
hashtags	[String]	Parsed from content, indexed for tag search

5.3 Comment

Field	Type	Notes
postId	ObjectId → Post	Required, the parent post
userId	ObjectId → User	Required, the comment author
content	String	Required, max 500 characters

5.4 Message

Field	Type	Notes
senderId	ObjectId → User	Required
receiverId	ObjectId → User	Required
content	String	Max 2000 characters, defaults to empty string

Field	Type	Notes
media	String	S3 URL of attached image/video
mediaType	String enum	'image' 'video' 'none'
isRead	Boolean	Default false; drives unread badge counts

5.5 Notification

Field	Type	Notes
recipientId	ObjectId → User	Required, who receives the notification
senderId	ObjectId → User	Required, who triggered it
type	String enum	'like' 'comment' 'follow' 'message'
postId	ObjectId → Post	Nullable; set for like/comment notifications
isRead	Boolean	Default false

5.6 Entity Relationships

Entity A	Relationship	Entity B	Cardinality
User	authors	Post	One-to-Many
User	writes	Comment	One-to-Many
Post	has	Comment	One-to-Many
User	likes	Post	Many-to-Many (embedded array)
User	follows	User	Many-to-Many (self-referential)
User	sends/receives	Message	Many-to-Many (via sender/receiver pairs)
User	triggers/receives	Notification	Many-to-Many (via sender/recipient)
Post	referenced by	Notification	One-to-Many, optional

6. Core Features

6.1 Authentication

- Register and log in with either email or username, plus password.
- JWT-based session, persisted client-side and attached to every API request.
- Protected routes redirect unauthenticated users to the login screen; invalid or expired tokens trigger an automatic logout.

6.2 Home Feed & Explore

- Infinite-scroll home feed showing posts from followed users, newest first.
- Explore tab with Latest and Trending sort modes and hashtag filtering.

- Skeleton loading states and empty-state illustrations for a polished first paint.

6.3 Posts, Likes & Comments

- Create posts with text, an image, or a video attachment.
- Like/unlike with optimistic UI feedback; Instagram-style double-tap-to-like on media.
- Threaded comments per post, with delete permissions for the comment author or the post owner.
- Hashtags in post text are automatically parsed into clickable links into Explore.
- Post owners can delete their own posts.

6.4 Profiles & Social Graph

- Editable bio, profile picture, and cover photo.
- Follow / unfollow with a live follower/following count.
- Followers and following lists in a searchable modal, with inline follow/unfollow actions.
- Suggested-users panel and global user search.

6.5 Real-Time Messaging

- One-to-one direct messaging with text and media attachments.
- Live typing indicators and read receipts.
- Online/offline presence with "last seen" timestamps.
- Message and full-conversation deletion, confirmed via an in-app dialog rather than a native browser prompt.

6.6 Notifications

- Real-time notifications for likes, comments, and new followers, delivered over Socket.IO.
- Unread badge counts in the navigation, cleared automatically when the relevant page is opened.
- Mark-all-as-read action.

6.7 Feedback System

- A centralized toast system (success / error / warning / info) surfaces feedback for every user-facing action across the app.
- Destructive actions (e.g. deleting a conversation) are confirmed through a custom in-app dialog, replacing native `window.confirm()` prompts.

6.8 Responsive Design

- Dedicated mobile layout: collapsible bottom navigation bar and a compact mobile header, in place of the desktop sidebar.
- A distinct brand identity — custom logo mark and favicon — applied consistently across the login screen, sidebar, and mobile header.

7. API Reference

All routes are mounted under `/api`. Every endpoint other than `register` and `login` requires a valid Bearer JWT, enforced by the `protect` middleware.

7.1 Auth — `/api/auth`

Method	Endpoint	Description
POST	<code>/register</code>	Create an account (validated username/email/password)
POST	<code>/login</code>	Authenticate with email-or-username + password, returns a JWT
GET	<code>/me</code>	Return the currently authenticated user

7.2 Users — `/api/users`

Method	Endpoint	Description
GET	<code>/search?q=</code>	Search users by username
GET	<code>/suggestions</code>	Suggested users to follow
GET	<code>/profile/:username</code>	Fetch a public profile
GET	<code>/:id</code>	Fetch a user by ID
PUT	<code>/profile</code>	Update bio / profile picture / cover picture
PUT	<code>/:id/follow</code>	Follow a user
PUT	<code>/:id/unfollow</code>	Unfollow a user

7.3 Posts — `/api/posts`

Method	Endpoint	Description
GET	<code>/feed</code>	Paginated feed of followed users' posts
GET	<code>/explore</code>	Paginated public posts (latest / trending, optional tag filter)
GET	<code>/user/:username</code>	All posts by a given user
POST	<code>/</code>	Create a post (optional media upload)
DELETE	<code>/:id</code>	Delete a post (owner only)
PUT	<code>/:id/like</code>	Toggle like on a post
POST	<code>/:postId/comments</code>	Add a comment
GET	<code>/:postId/comments</code>	List comments on a post
DELETE	<code>/:postId/comments/:id</code>	Delete a comment (author or post owner)

7.4 Messages — `/api/messages`

Method	Endpoint	Description
GET	<code>/conversations/list</code>	List all conversations with last-message previews
POST	<code>/</code>	Send a message (optional media upload)

Method	Endpoint	Description
GET	/:userId	Fetch message history with a given user
DELETE	/:id	Delete a single message
DELETE	/conversation/:userId	Delete an entire conversation

7.5 Notifications — /api/notifications

Method	Endpoint	Description
GET	/	List notifications for the current user
PUT	/read	Mark all notifications as read

8. Real-Time Events (Socket.IO)

Alongside the REST API, Oku maintains a persistent WebSocket connection per client for low-latency delivery of messages, notifications, typing indicators, and presence updates.

Event	Direction	Purpose
setup	Client → Server	Registers the connected socket against a user ID and marks the user online
onlineUsers	Server → All Clients	Broadcasts the current list of online user IDs
joinChat	Client → Server	Joins a deterministic two-user chat room
typing / stopTyping	Bidirectional	Relays typing-indicator state within a chat room
receiveMessage	Server → Client	Delivers a newly sent message in real time
receiveNotification	Server → Client	Delivers a new like / comment / follow notification
messageDeleted	Server → Client	Syncs a message deletion across open sessions
conversationDeleted	Server → Client	Syncs a full conversation deletion across open sessions
disconnect	Client → Server	Marks the user offline and records lastSeen

9. Security Measures

- Passwords are hashed with bcrypt before storage and are never included in API responses (stripped via a custom Mongoose toJSON method).
- Authentication is stateless via signed JWTs; a protect middleware verifies the token on every protected route.
- express-validator enforces input validation on all mutating endpoints (registration, login, posts, comments, messages).
- express-rate-limit applies a general API limiter (300 requests / 15 min) and a stricter authentication limiter (10 attempts / 15 min) to blunt brute-force and credential-stuffing attempts.
- Helmet sets secure HTTP response headers, tuned to still permit cross-origin media loading for post/message attachments.

- File uploads are restricted by MIME type and extension (images/video only) and capped at 25MB before they ever reach cloud storage.
- CORS is explicitly configured at the API boundary.

10. Challenges & Solutions

Challenge	Solution
Keeping REST-fetched state and real-time WebSocket events consistent (unread badges, conversation lists, notification lists)	A shared Socket context feeds targeted state updates into page-level state on each relevant socket event, rather than re-fetching from the API
Distinguishing a single tap (open media) from a double tap (like) without the browser's slower native dblclick event	A manual click-timer pattern buffers the first tap briefly to detect a following second tap
Native browser dialogs (window.confirm) breaking visual and theming consistency	Replaced with a custom, themed ConfirmDialog component for destructive actions
Errors were only logged to the console, giving users no feedback on failed actions	Introduced a centralized toast notification system wired into every API interaction across the app

11. Future Enhancements

- Browser and mobile push notifications
- Post bookmarking / saved posts
- Hashtag trending analytics dashboard
- End-to-end encrypted direct messages
- Admin moderation tooling, building on the existing isAdmin flag
- Automated unit and integration test coverage

12. Conclusion

Oku delivers a complete, production-shaped social media platform: secure authentication, a media-rich content model, a social graph with follows and a personalized feed, and real-time messaging and notifications backed by Socket.IO. The MERN stack, combined with cloud object storage for media and a layered security posture (hashing, rate limiting, validation, secure headers), provides a solid, extensible foundation for the future enhancements outlined above.